

Implementación Patrón Builder



**Universidad
del Cauca**

Vigilada Mineducación

Presentado por:

Brayan Stiven Charry Vela

Jhoiner Alberto Puentes Figueroa

Sebastian Ruiz Segura

Profesor:

Roberto Encarnacion Mosquera

Universidad del Cauca

Ingeniería electrónica y telecomunicaciones Ingeniería de sistemas

Popayán, mayo

1. Introducción

En este informe se documenta la implementación del patrón de diseño creacional Builder dentro del proyecto Sistema PiedraAzul, específicamente en el módulo de agenda de citas. La mejora se centra en la construcción de objetos CitaResponse a partir de la entidad Cita, aplicando una clase abstracta, herencia, un builder concreto y un director de construcción.

La implementación evita que el controlador REST tenga toda la responsabilidad de armar manualmente la respuesta de una cita. De esta forma se mejora la organización del código, se facilita el mantenimiento y se cumple con los criterios del taller.

2. Objetivo del taller

Implementar el patrón de diseño creacional Builder para modelar la creación de citas en el proyecto del curso, evidenciando el uso de clases abstractas, herencia, separación de responsabilidades y pruebas unitarias.

3. Definiciones principales

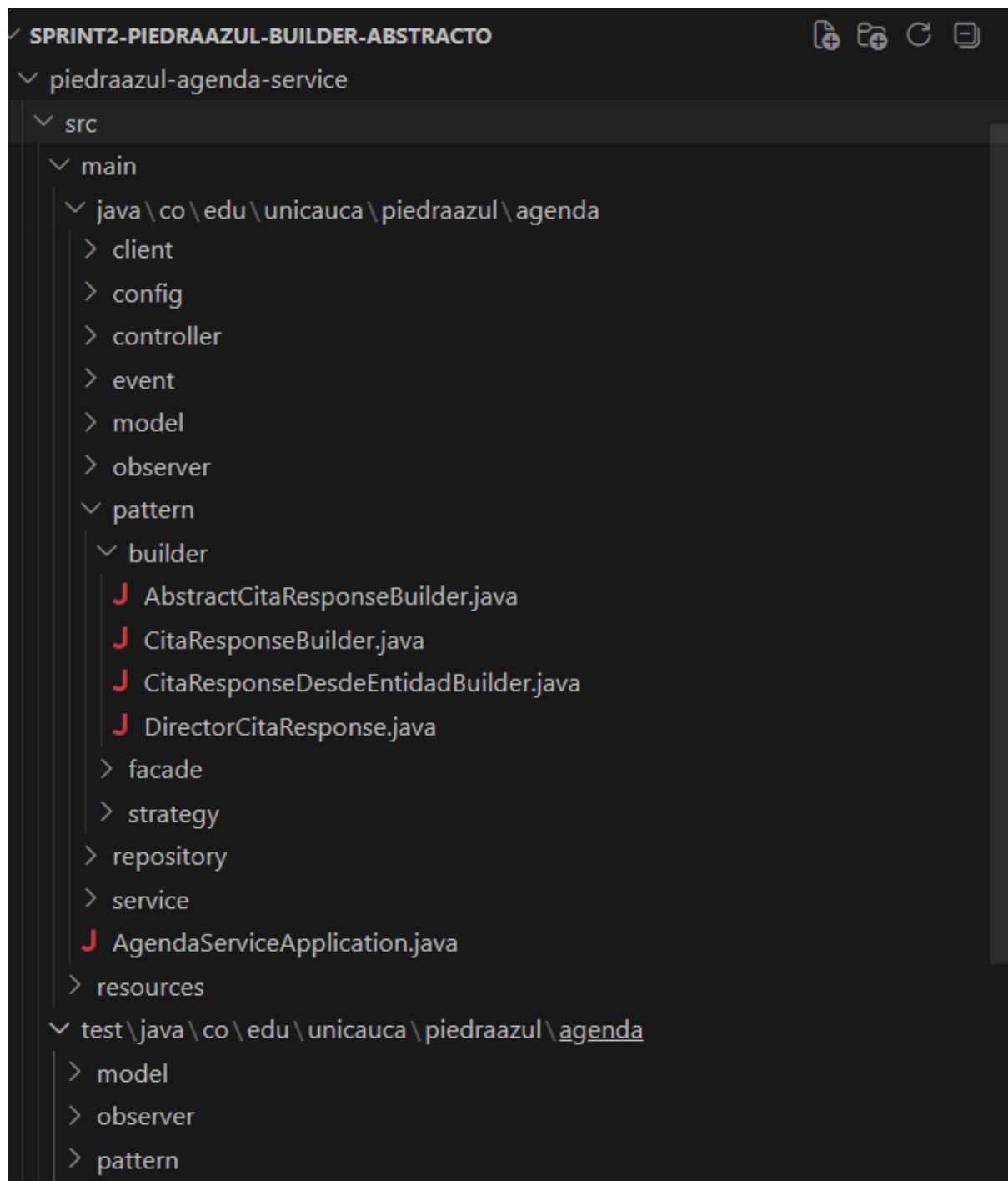
Concepto	Definición aplicada al proyecto
Patrón Builder	Es un patrón creacional que permite construir objetos complejos paso a paso. Separa el proceso de construcción del objeto final, de modo que el mismo proceso pueda generar distintas representaciones.
Producto	Es el objeto final que se construye. En este caso, el producto es CitaResponse, que representa la respuesta que entrega el servicio de agenda cuando se consulta o crea una cita.
Builder abstracto	Es una clase abstracta que define los pasos obligatorios para construir el producto. En el proyecto corresponde a AbstractCitaResponseBuilder.

Builder concreto	Es la clase que hereda del builder abstracto e implementa cada paso de construcción. En el proyecto corresponde a CitaResponseDesdeEntidadBuilder.
Director	Es la clase que controla el orden en que se ejecutan los pasos de construcción. En el proyecto corresponde a DirectorCitaResponse.
Cliente	Es la clase que utiliza el director y el builder concreto. En esta implementación, el cliente es CitaRestController.
Herencia	Mecanismo de programación orientada a objetos mediante el cual una clase hija reutiliza y especializa el comportamiento definido por una clase padre.
Separación de responsabilidades	Principio que busca que cada clase tenga una responsabilidad clara. En este caso, el controlador atiende peticiones, el builder arma el DTO y el director ordena la construcción.

4. Evidencia de la estructura del proyecto

La implementación se realizó dentro del módulo piedraazul-agenda-service. Las clases del patrón se ubicaron en el paquete pattern.builder y las pruebas unitarias en el paquete equivalente dentro de src/test.

Figura 1. Estructura de carpetas y clases relacionadas con el patrón Builder.

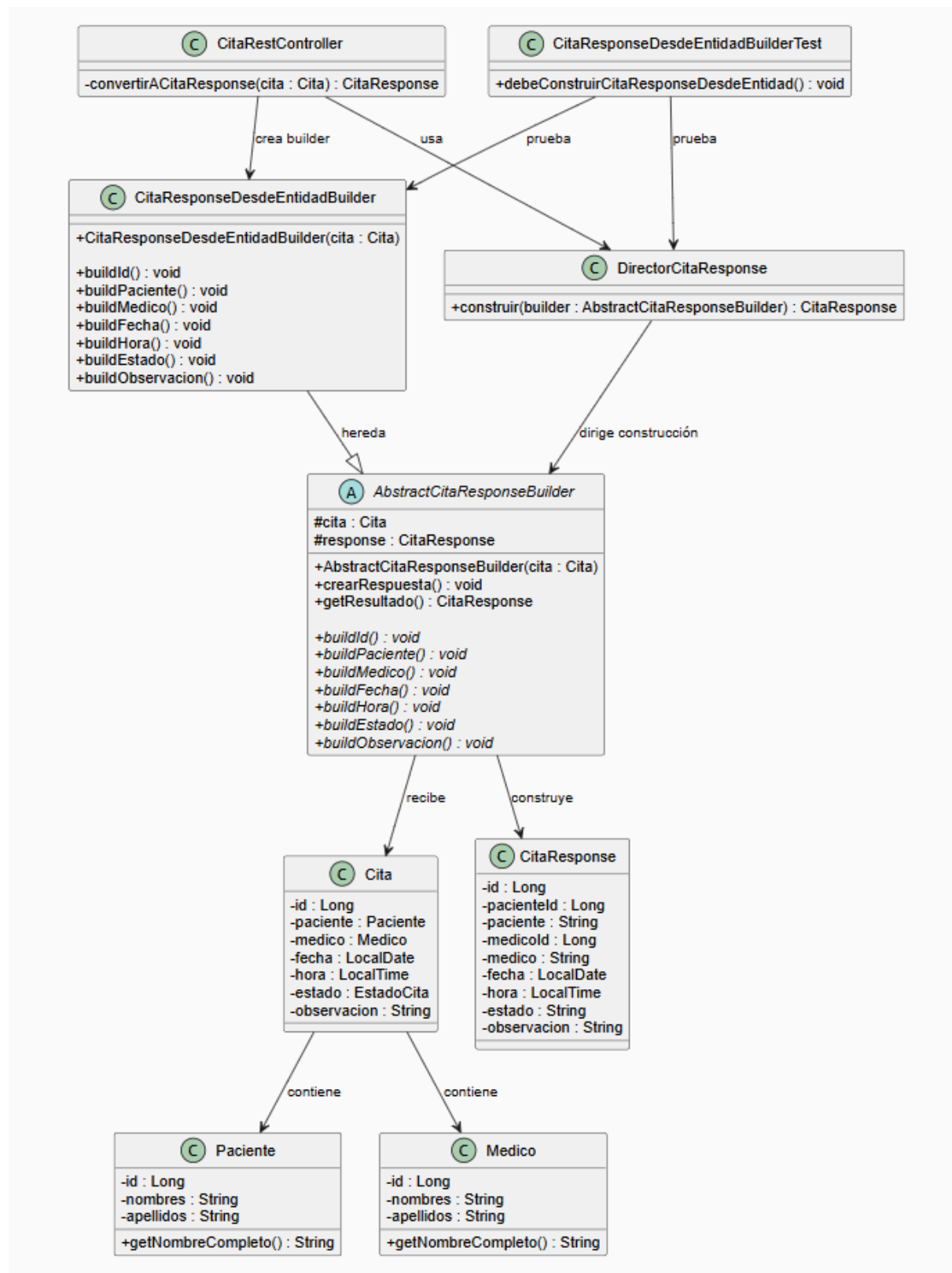


5. Diseño de la solución

La solución toma una entidad Cita, extrae sus datos principales y construye un objeto CitaResponse. Para esto, el proceso se distribuye entre las siguientes clases:

- AbstractCitaResponseBuilder: define los pasos generales de construcción.
- CitaResponseDesdeEntidadBuilder: implementa los pasos concretos para construir la respuesta desde una entidad Cita.
- DirectorCitaResponse: ejecuta los pasos en un orden controlado.
- CitaRestController: usa el director y el builder para retornar la respuesta de la API.

Figura 2. Diagrama de clases del patrón Builder aplicado a CitaResponse.

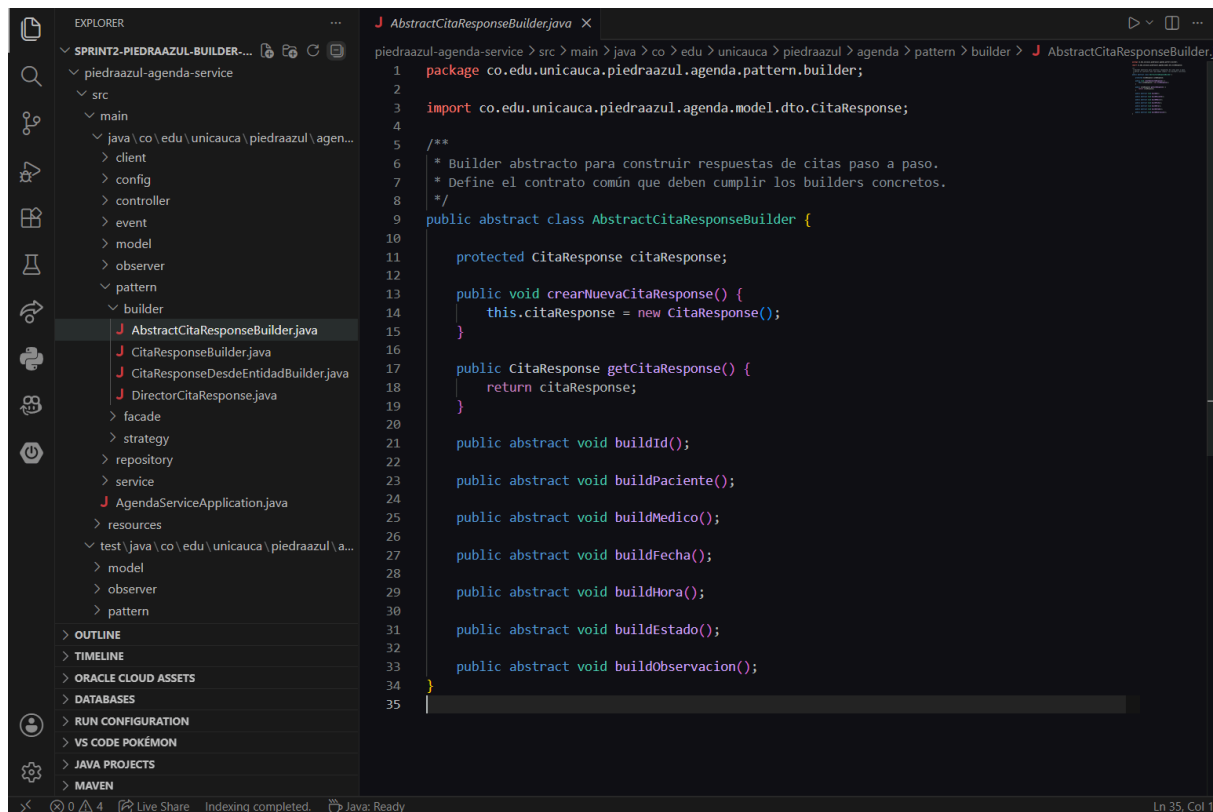


6. Implementación del Builder abstracto

La clase AbstractCitaResponseBuilder representa el builder abstracto. En ella se declara el producto que se va a construir y se definen los métodos abstractos que toda implementación concreta debe completar.

- Atributo protegido citaResponse: almacena temporalmente el producto en construcción.
- crearNuevaCitaResponse(): inicializa el objeto que será construido.
- getCitaResponse(): retorna el producto final.
- Métodos buildId, buildPaciente, buildMedico, buildFecha, buildHora, buildEstado y buildObservacion: definen los pasos obligatorios de construcción.

Figura 3. Código de la clase abstracta AbstractCitaResponseBuilder.

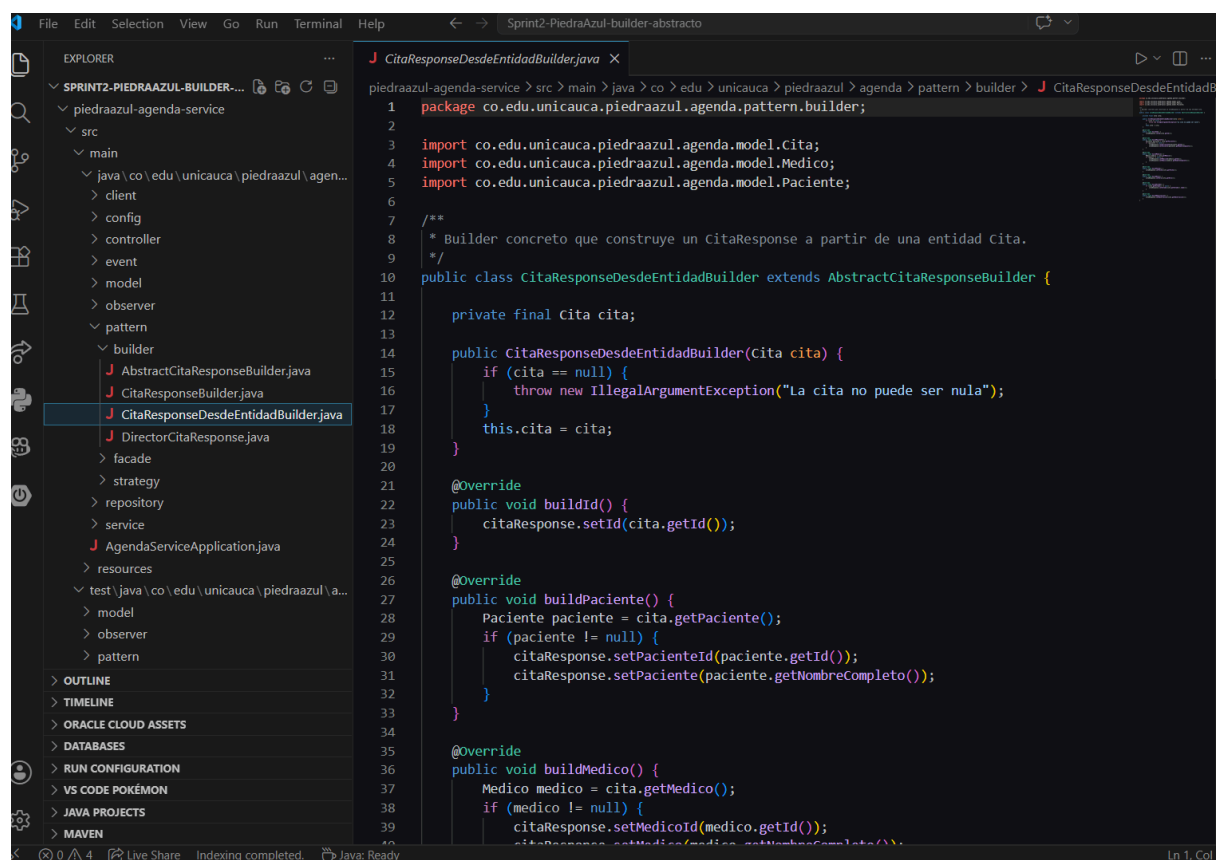


7. Implementación del Builder concreto

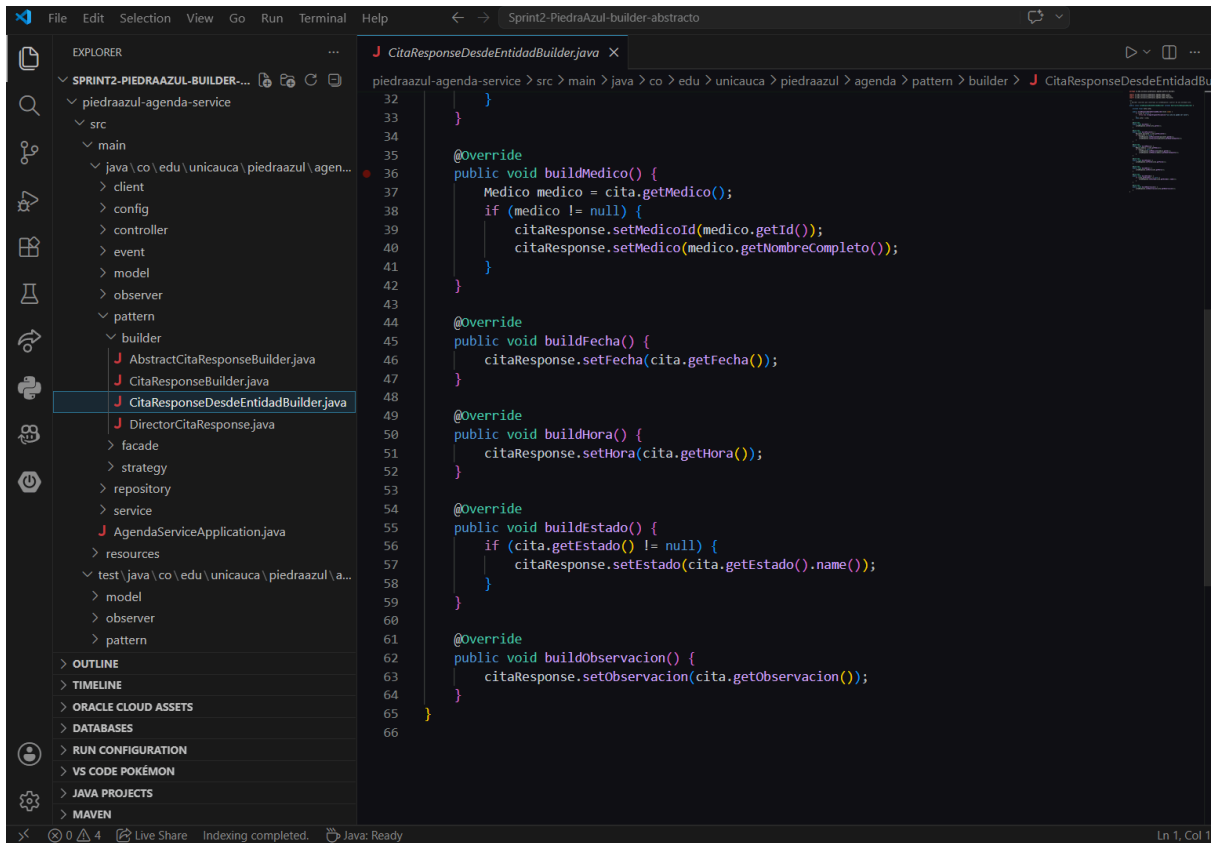
La clase `CitaResponseDesdeEntidadBuilder` hereda de `AbstractCitaResponseBuilder`. Su responsabilidad es construir un objeto `CitaResponse` tomando como fuente una entidad `Cita`. Esta clase demuestra el uso de herencia porque extiende el builder abstracto e implementa todos sus métodos.

Además, valida que la cita no sea nula y verifica la existencia del paciente y del médico antes de asignar sus datos al DTO de respuesta.

Figura 4. Código del builder concreto `CitaResponseDesdeEntidadBuilder`.



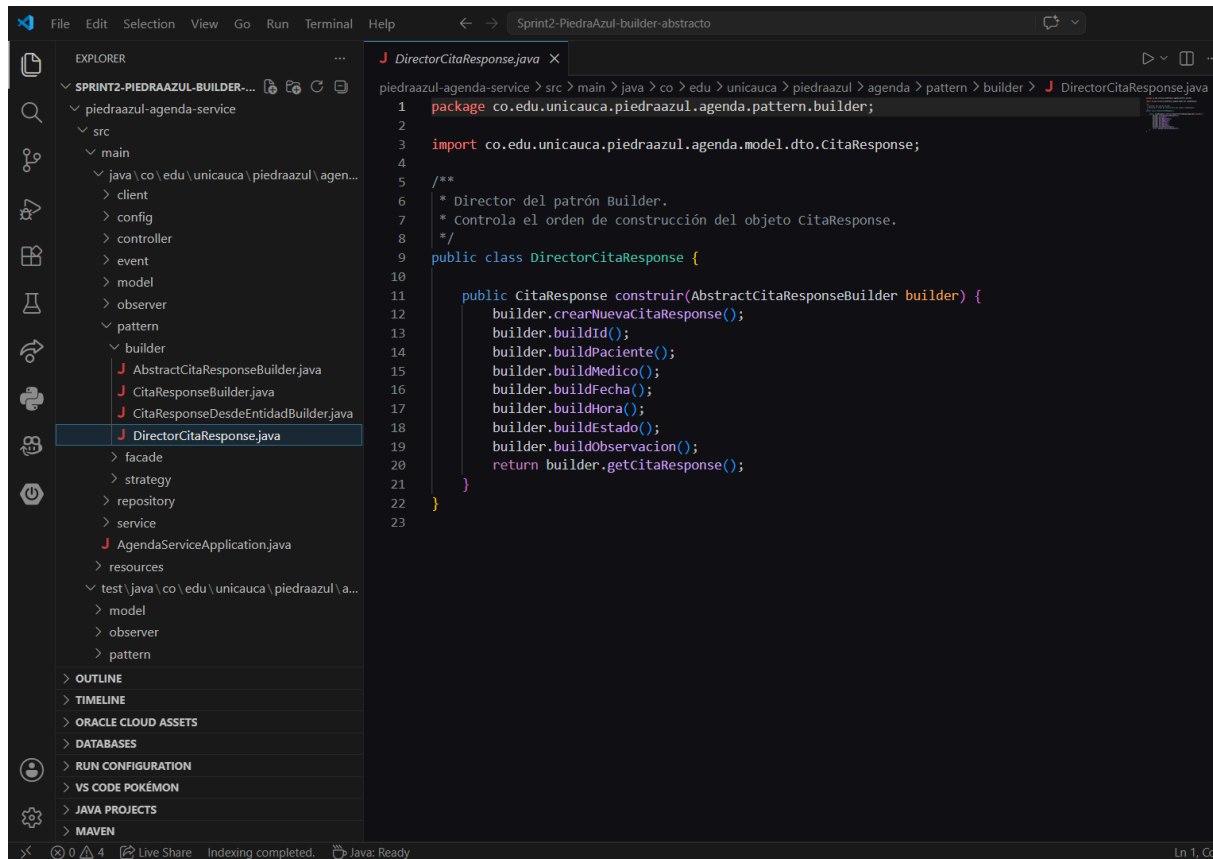
```
1 package co.edu.unicauca.piedraazul.agenda.pattern.builder;
2
3 import co.edu.unicauca.piedraazul.agenda.model.Cita;
4 import co.edu.unicauca.piedraazul.agenda.model.Medico;
5 import co.edu.unicauca.piedraazul.agenda.model.Paciente;
6
7 /**
8  * Builder concreto que construye un CitaResponse a partir de una entidad Cita.
9  */
10 public class CitaResponseDesdeEntidadBuilder extends AbstractCitaResponseBuilder {
11
12     private final Cita cita;
13
14     public CitaResponseDesdeEntidadBuilder(Cita cita) {
15         if (cita == null) {
16             throw new IllegalArgumentException("La cita no puede ser nula");
17         }
18         this.cita = cita;
19     }
20
21     @Override
22     public void buildId() {
23         citaResponse.setId(cita.getId());
24     }
25
26     @Override
27     public void buildPaciente() {
28         Paciente paciente = cita.getPaciente();
29         if (paciente != null) {
30             citaResponse.setPacienteId(paciente.getId());
31             citaResponse.setPaciente(paciente.getNombreCompleto());
32         }
33     }
34
35     @Override
36     public void buildMedico() {
37         Medico medico = cita.getMedico();
38         if (medico != null) {
39             citaResponse.setMedicoId(medico.getId());
40             citaResponse.setMedico(medico.getNombreCompleto());
41         }
42     }
43 }
```

8. Implementación del Director

La clase `DirectorCitaResponse` corresponde al Director del patrón Builder. Su función es recibir un builder abstracto y ejecutar los pasos de construcción en un orden definido. Esto permite que el proceso de creación quede centralizado y que el controlador no conozca los detalles internos de cada paso.

Figura 5. Código del `DirectorCitaResponse`.



The screenshot shows a code editor with the following Java code for `DirectorCitaResponse.java`:

```
1 package co.edu.unicauca.piedraazul.agenda.pattern.builder;
2
3 import co.edu.unicauca.piedraazul.agenda.model.dto.CitaResponse;
4
5 /**
6  * Director del patrón Builder.
7  * Controla el orden de construcción del objeto CitaResponse.
8  */
9 public class DirectorCitaResponse {
10
11     public CitaResponse construir(AbstractCitaResponseBuilder builder) {
12         builder.crearNuevaCitaResponse();
13         builder.buildId();
14         builder.buildPaciente();
15         builder.buildMedico();
16         builder.buildFecha();
17         builder.buildHora();
18         builder.buildEstado();
19         builder.buildObservacion();
20         return builder.getCitaResponse();
21     }
22 }
23
```

The left sidebar shows the project structure with the following folders and files:

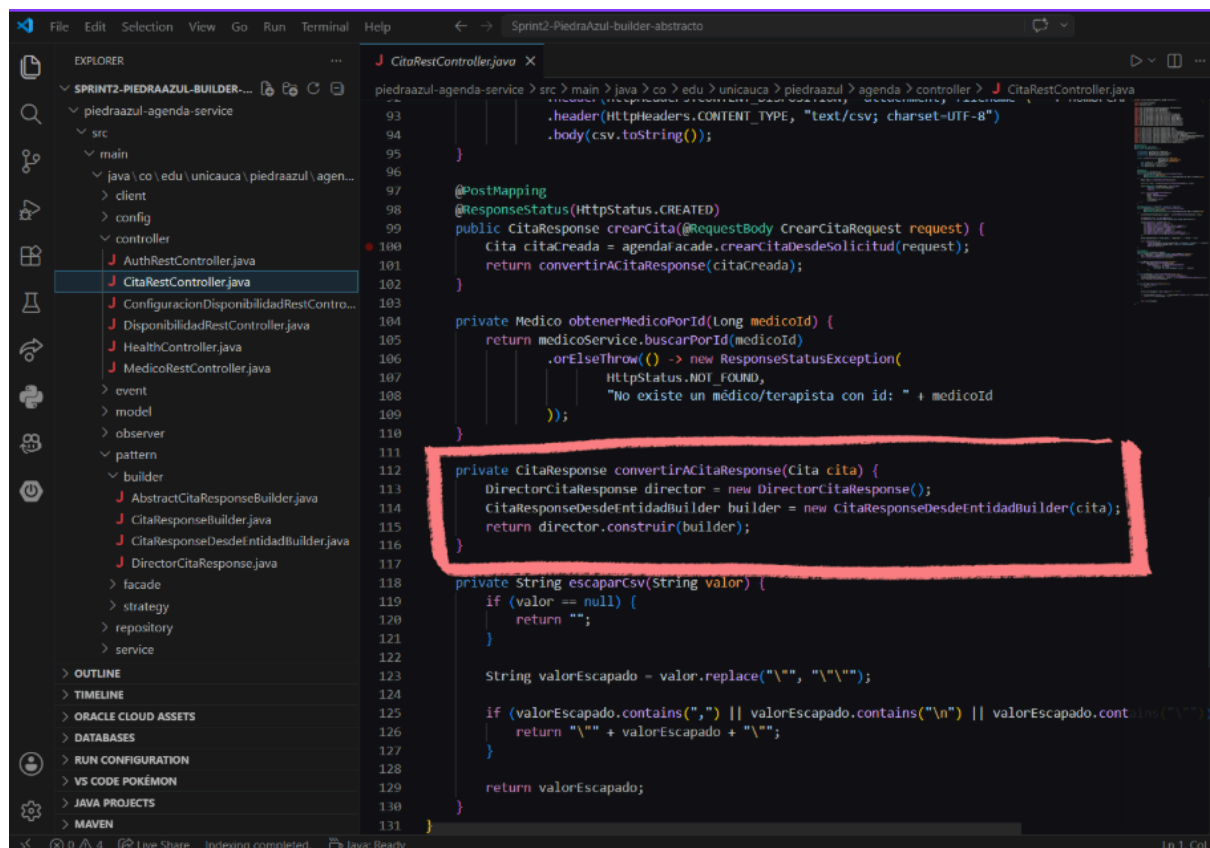
- SPRINT2-PIEDRAAZUL-BUILDER-...
- ▼ piedraazul-agenda-service
 - ▼ src
 - ▼ main
 - ▼ java \co \edu \unicauca \piedraazul \agen...
 - > client
 - > config
 - > controller
 - > event
 - > model
 - > observer
 - > pattern
 - ▼ builder
 - AbstractCitaResponseBuilder.java
 - CitaResponseBuilder.java
 - CitaResponseDesdeEntidadBuilder.java
 - DirectorCitaResponse.java
 - > facade
 - > strategy
 - > repository
 - > service
 - AgendaServiceApplication.java
 - > resources
 - ▼ test \java \co \edu \unicauca \piedraazul \a...
 - > model
 - > observer
 - > pattern- ▼ OUTLINE
- ▼ TIMELINE
- ▼ ORACLE CLOUD ASSETS
- ▼ DATABASES
- ▼ RUN CONFIGURATION
- ▼ VS CODE POKÉMON
- ▼ JAVA PROJECTS
- ▼ MAVEN

9. Integración con el controlador REST

El patrón no quedó como un ejemplo aislado. Fue integrado en CitaRestController, dentro del método `convertirACitaResponse`. Antes, el controlador construía directamente el `CitaResponse`; ahora delega esa tarea al `DirectorCitaResponse` y al `CitaResponseDesdeEntidadBuilder`.

Con esto se evidencia la separación de responsabilidades: el controlador atiende la petición HTTP, el builder construye el DTO y el director controla el flujo de construcción.

Figura 6. Uso del Builder desde CitaRestController.

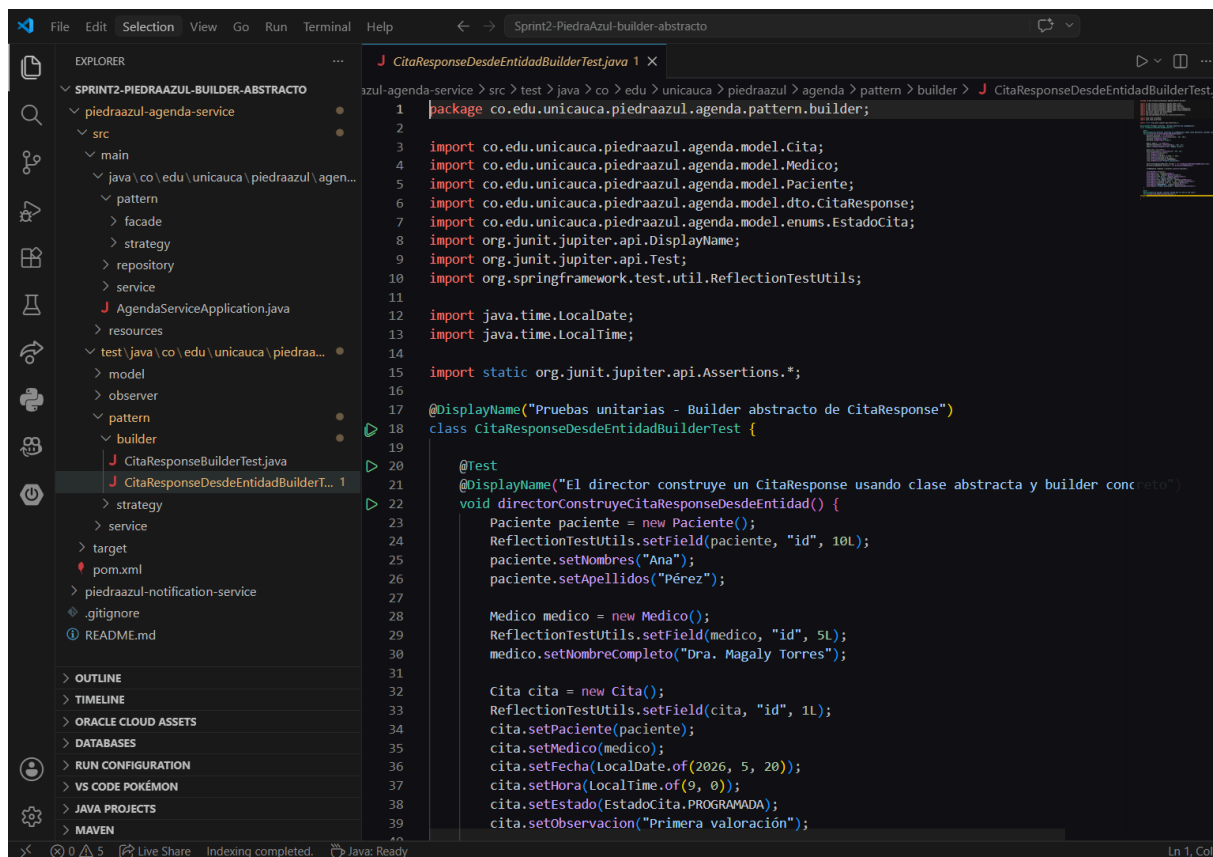


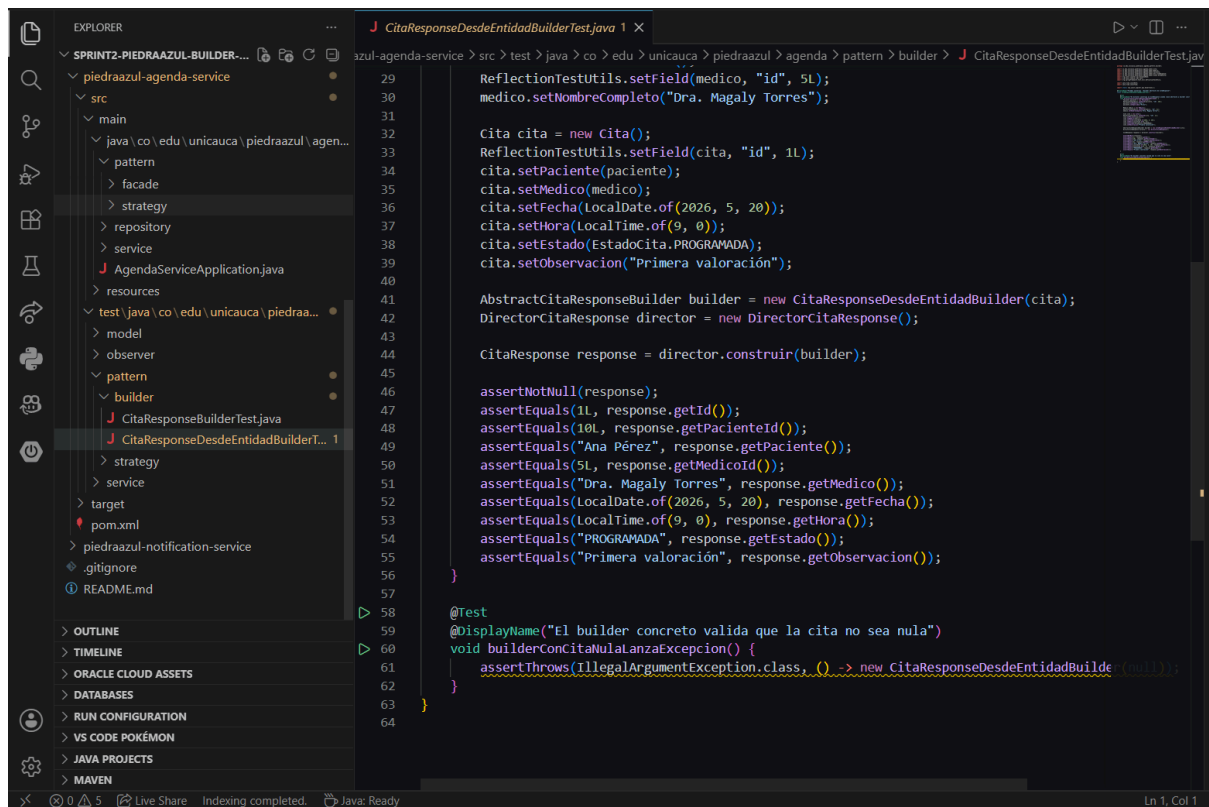
10. Pruebas unitarias

Para validar la implementación se agregó una prueba unitaria llamada `CitaResponseDesdeEntidadBuilderTest`. Esta prueba crea objetos `Paciente`, `Médico` y `Cita`, luego utiliza `AbstractCitaResponseBuilder`, `CitaResponseDesdeEntidadBuilder` y `DirectorCitaResponse` para construir la respuesta y verificar sus valores con aserciones.

También se valida que el builder concreto lance una excepción cuando recibe una cita nula, evitando construcciones inválidas.

Figura 7. Prueba unitaria del Builder abstracto.



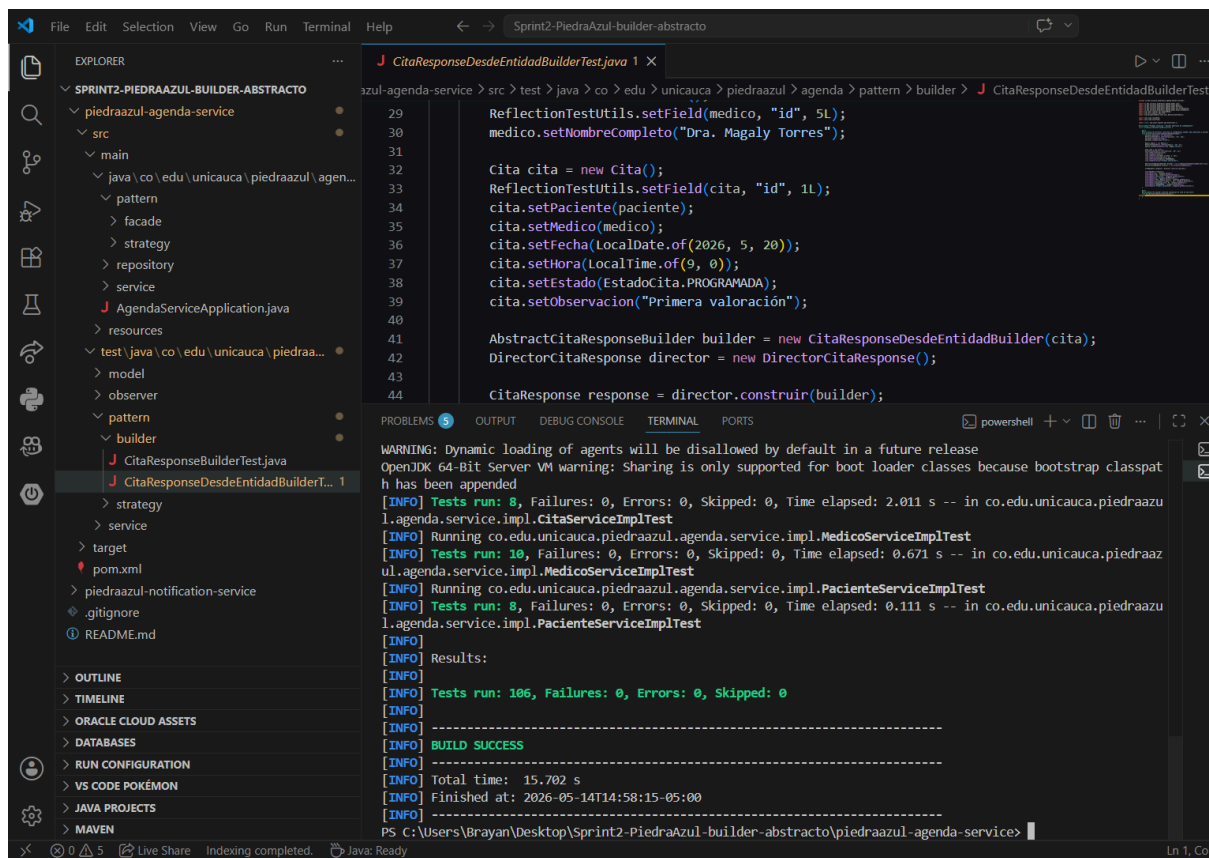


```
29 ReflectionTestUtils.setField(medico, "id", 5L);
30 medico.setNombreCompleto("Dra. Magaly Torres");
31
32 Cita cita = new Cita();
33 ReflectionTestUtils.setField(cita, "id", 1L);
34 cita.setPaciente(paciente);
35 cita.setMedico(medico);
36 cita.setFecha(LocalDate.of(2026, 5, 20));
37 cita.setHora(LocalTime.of(9, 0));
38 cita.setEstado(EstadoCita.PROGRAMADA);
39 cita.setObservacion("Primera valoración");
40
41 AbstractCitaResponseBuilder builder = new CitaResponseDesdeEntidadBuilder(cita);
42 DirectorCitaResponse director = new DirectorCitaResponse();
43
44 CitaResponse response = director.construir(builder);
45
46 assertNotNull(response);
47 assertEquals(1L, response.getId());
48 assertEquals(10L, response.getPacienteId());
49 assertEquals("Ana Pérez", response.getPaciente());
50 assertEquals(5L, response.getMedicoId());
51 assertEquals("Dra. Magaly Torres", response.getMedico());
52 assertEquals(LocalDate.of(2026, 5, 20), response.getFecha());
53 assertEquals(LocalTime.of(9, 0), response.getHora());
54 assertEquals("PROGRAMADA", response.getEstado());
55 assertEquals("Primera valoración", response.getObservacion());
56
57
58
59
60 @Test
61 @DisplayName("El builder concreto valida que la cita no sea nula")
62 void builderConCitaNulaLanzaExcepcion() {
63     // assertThrows(IllegalArgumentException.class, () -> new CitaResponseDesdeEntidadBuilder(null));
64 }
```

Ejecución del test, con la siguiente cláusula:

mvn test

Figura 8. Espacio para reemplazar por el pantallazo real de mvn test.



11. Conclusiones

Con el desarrollo de este taller se logró implementar y fortalecer el patrón de diseño creacional Builder dentro del proyecto PiedraAzul, específicamente en el módulo piedraazul-agenda-service. La mejora realizada permitió organizar de mejor manera la construcción de objetos CitaResponse, los cuales representan la información que el sistema entrega como respuesta cuando se consultan o gestionan citas médicas. Inicialmente, el proyecto ya contaba con una implementación básica del patrón Builder mediante una clase fluente. Sin embargo, para cumplir de forma más completa con los criterios de evaluación del taller, se incorporó una estructura más formal del patrón, basada en una clase abstracta, un builder concreto y una clase directora. Esta modificación permitió aplicar herencia, abstracción y separación de responsabilidades, aspectos fundamentales dentro del diseño orientado a objetos. La clase AbstractCitaResponseBuilder define los pasos generales para construir una respuesta de cita, mientras que CitaResponseDesdeEntidadBuilder se encarga de implementar esos pasos tomando la información desde una entidad Cita. Por su

parte, DirectorCitaResponse controla el orden en que se ejecuta el proceso de construcción. Gracias a esta distribución, el controlador CitaRestController ya no tiene la responsabilidad directa de construir manualmente el objeto de respuesta, sino que delega esta tarea al Builder, haciendo que el código sea más limpio, mantenible y fácil de extender. Esta solución también permite que, en el futuro, el sistema pueda crear diferentes representaciones de una cita sin modificar la lógica principal del controlador. Por ejemplo, podrían agregarse nuevos builders para construir respuestas resumidas, respuestas detalladas, respuestas para reportes o respuestas adaptadas a otros módulos del sistema. Esto demuestra una ventaja importante del patrón Builder: separar el proceso de construcción del objeto final, facilitando la evolución del software. Además, se incluyeron pruebas unitarias para verificar que el Builder construye correctamente los objetos CitaResponse a partir de una entidad Cita. Estas pruebas permiten comprobar que los datos principales, como el identificador de la cita, paciente, médico, fecha, hora, estado y observación, son asignados correctamente durante el proceso de construcción.